Foundations

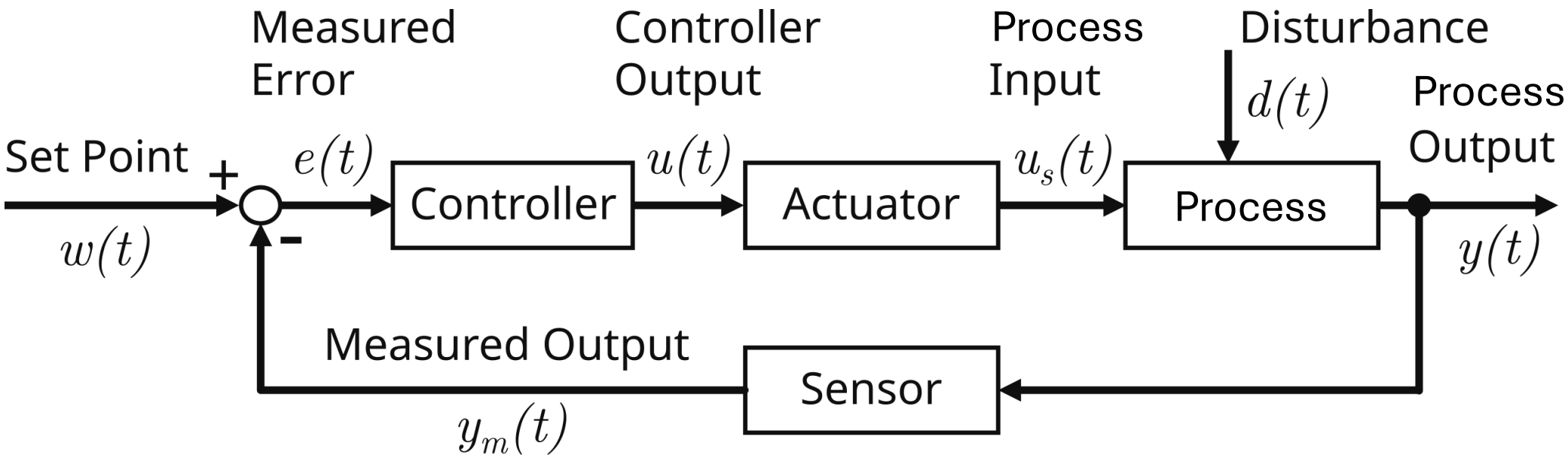
Reinforcement Learning

Motivation

Closed-Loop Control

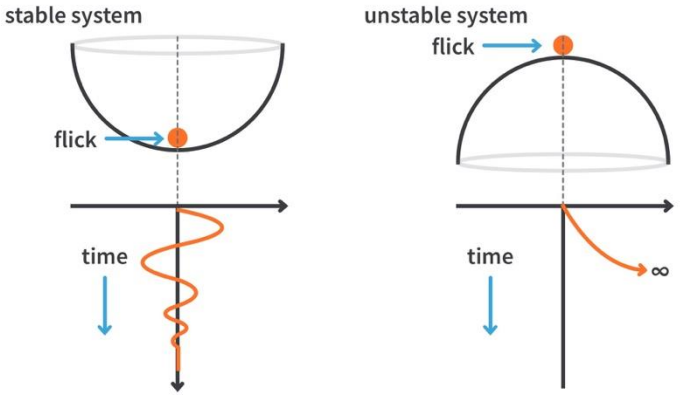
- controller compares reference and output
- error signal drives control action

maintains temperature near setpoint

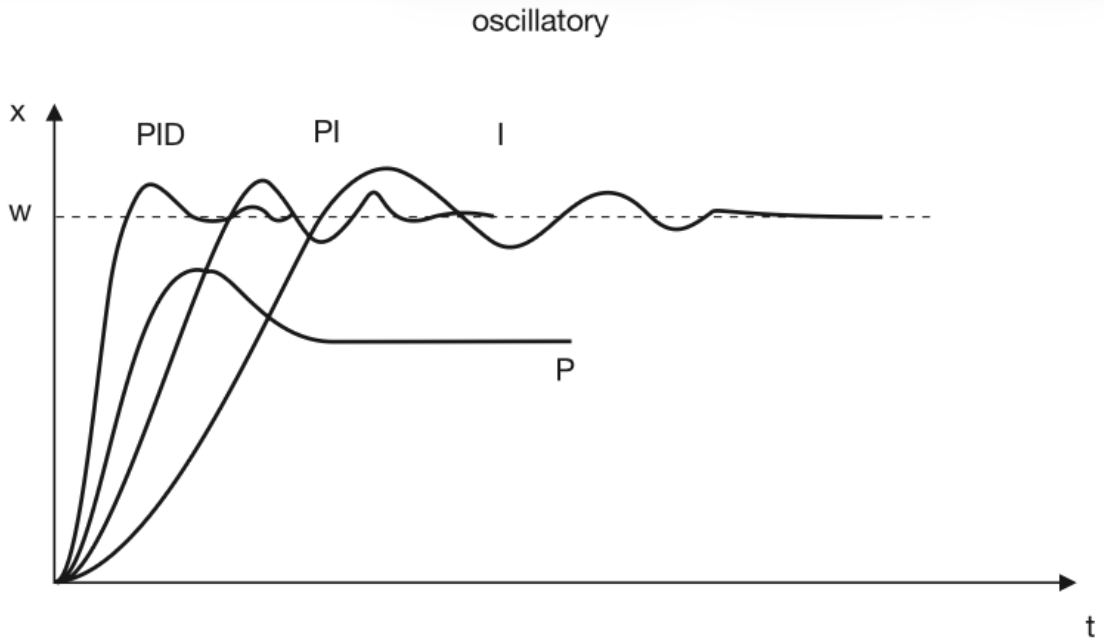


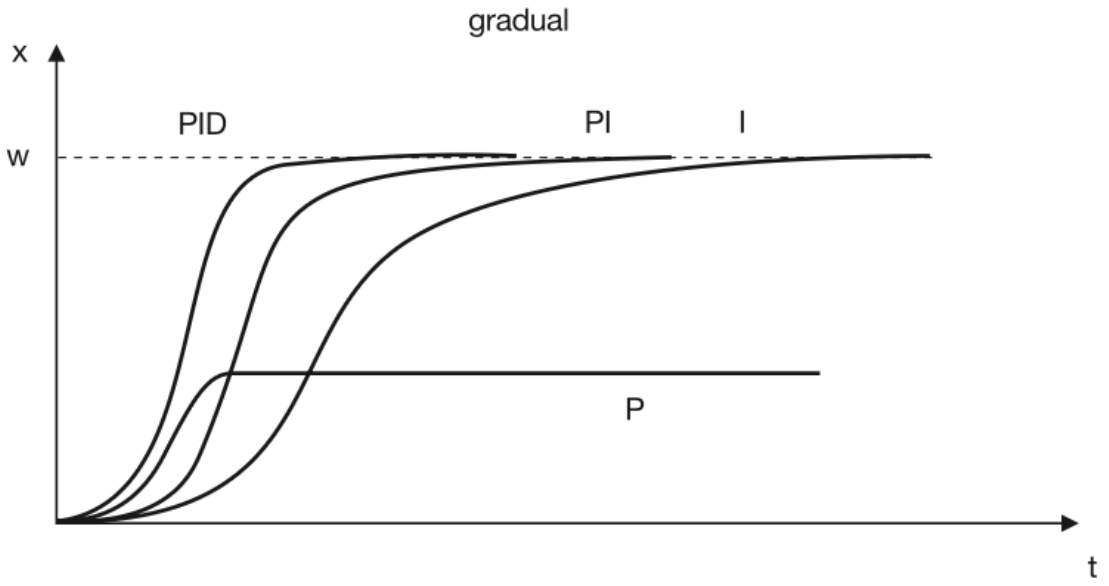
Control Objectives

- stability (bounded output)

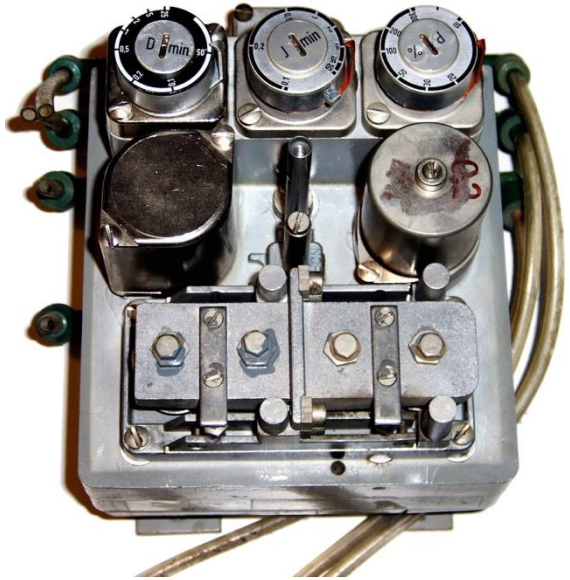


- fast response: rise and settling time
- low overshoot
- small steady-state error
- robustness (insensitivity to model uncertainty and disturbances)



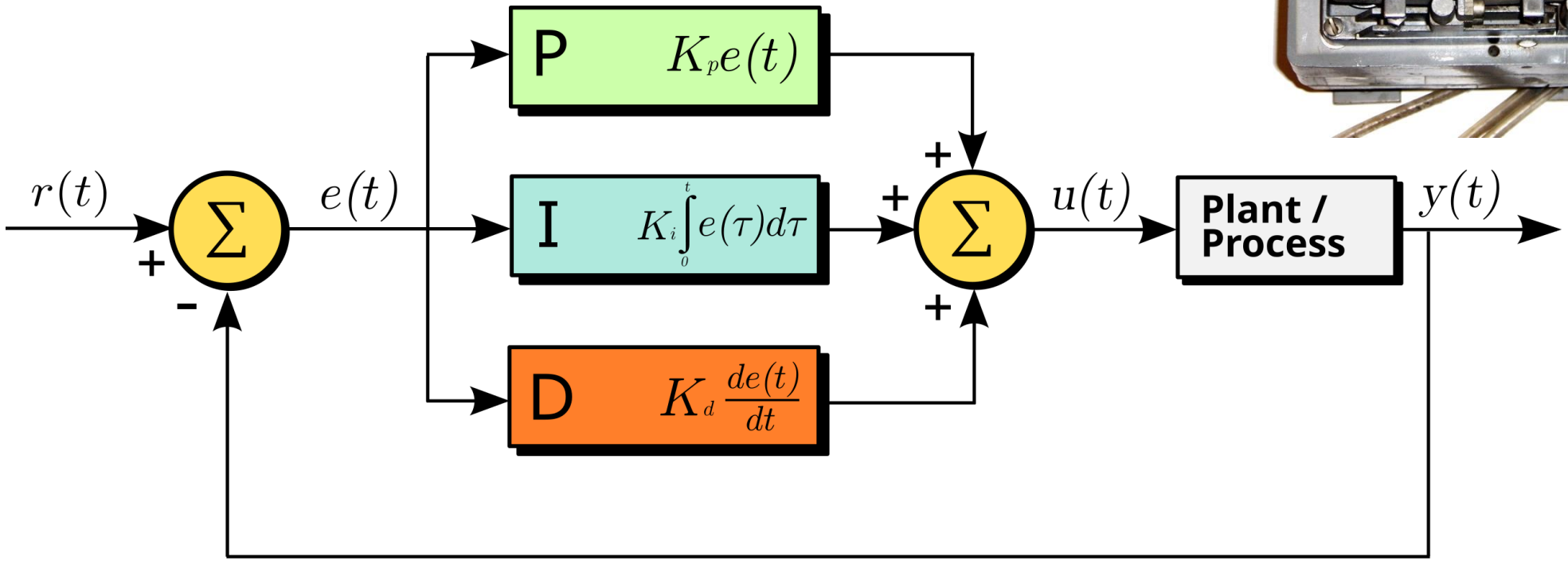


example: pneumatic PID



PID Controllers

most widely used industrial controller



purely reactive

Limits of Classic Control

- mostly reactive, tuned for simple linear systems
- poor handling of constraints and delays
- struggles with high-dimensional, nonlinear, or unstructured environments

many modern applications (robotics, autonomous driving, games) involve:

- large state/action spaces
- partial observability
- complex interactions

→ hard or impossible to design classic controllers manually

Model Predictive Control (MPC)

optimal control method

1. apply a model of the system to predict future developments (e.g., differential equation, but potentially also using ML)
2. adjust the manipulated variable to minimize the deviation of the controlled variable over several steps → optimization
3. execute only the first step, then repeat the process

requires model of the system

Model-Free vs Model-Based Control

- model-based control (MPC): uses a system model to plan or optimize actions
- model-free reinforcement learning (RL): learns control directly from interaction → no explicit system model needed

trade-off:

- model-based: sample efficient, predictive, naturally handles constraints
- model-free: flexible, works with unknown dynamics, maximizes long-term reward

When to Use What

rule of thumb:

- PID: simple loops
- MPC: constrained multivariable control
- RL: complex systems without good models

many modern systems combine them:

- PID for low-level control
- MPC for high-level trajectory optimization
- RL for strategy or policy learning

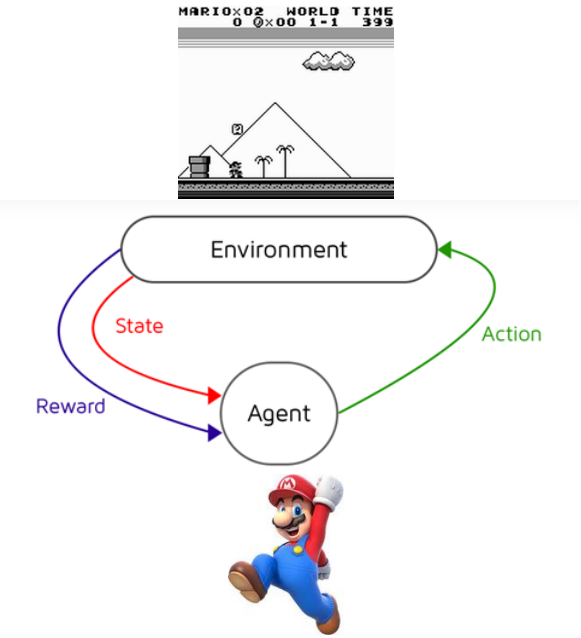
example: autonomous driving

- PID: steering torque
- MPC: trajectory tracking
- RL: driving behavior

Sequential Decision Making

reinforcement learning (RL):

formalization of sequential decision making (action policy) of software agent interacting with environment



corresponds to search for best (or rather good) action policy to reach a given goal (e.g., win a game)

using learning from examples (data) to guide the search

RL usually more difficult (e.g., non-differentiable as a whole) than supervised learning (which can be seen as “generalized optimization”, often of proxy metric)

Main Elements of RL

goal: find action policy maximizing reward from environment

action policy

- exploration-exploitation trade-off
- e.g., epsilon-greedy: random exploration at small fraction of the time

feedback from environment

- goal-directed, no supervision
- scalar reward signal, cumulative and delayed (credit assignment problem)

Optional Elements of RL

value functions for states or actions: improve efficiency of search in vast action policy space (alternative: direct policy search)

model of environment: (model-free) learning from trial-and-error or (model-based) planning

can be used in different ways:

- simulate experience from model (for learning)
- decision-time planning (e.g., heuristic search or model predictive control)

Mathematical Setup

Markov Decision Process (MDP)

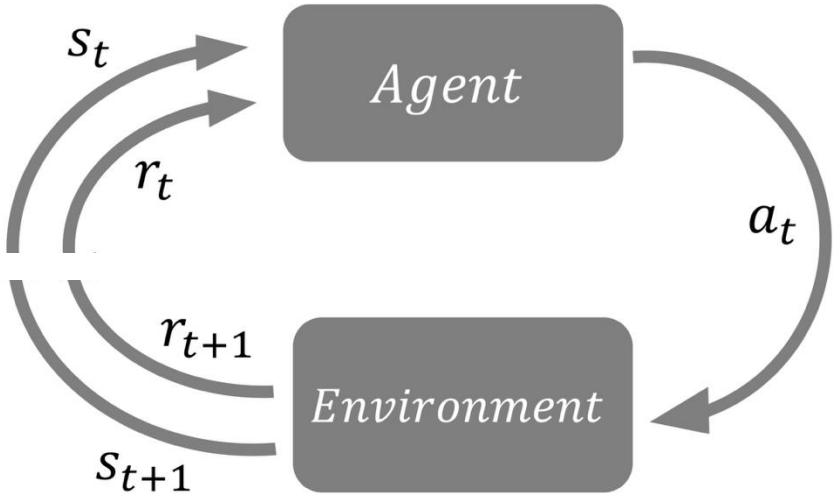
idea: current state includes all information about past

transition probabilities between states describe dynamics of given MDP

action policy: mapping from states to probabilities for selection of different actions

States, Actions, and Rewards

transition probabilities (model of environment): $p(s_{t+1}, r_{t+1} | s_t, a_t)$



reward hypothesis:

- reward as scalar signal
- goal: maximization of expected cumulative sum of received rewards

State and Action Values

state/action value: total amount of expected future reward starting from given state/action (usually with discounting of later steps)

→ indicating long-term desirability of states/actions

main motivation: improve efficiency of search in policy space

(for comparison: evolutionary methods search directly by evaluating entire policies)

State-Value Function

(needed for all states)

on-policy

return

discount rate

$$v_\pi(s_t) = E_\pi\left[\overbrace{\sum_{k=o}^{\infty} \gamma^k r_{t+k+1}}^{\text{return}} \,|s_t\right] = E_\pi[r_{t+1} + \gamma v_\pi(s_{t+1})|s_t]$$

$$= \sum_{a_t} \pi(a_t|s_t) \sum_{s'_{t+1}, r_{t+1}} p(s'_{t+1}, r_{t+1}|s_t, a_t)[r_{t+1} + \gamma v_\pi(s'_{t+1})]$$

policy: probabilitiy to take specific action being in a given state

transition probability (depending on environment) from state s_t to state s'_{t+1} for a given action

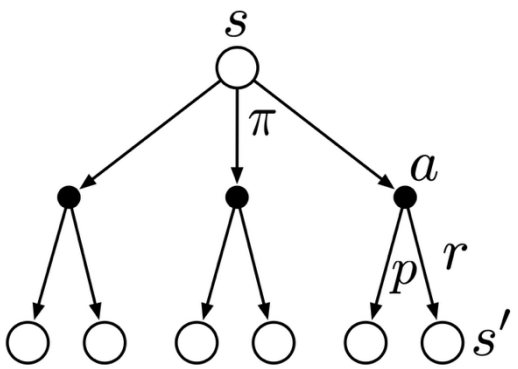
(sweep through entire state space)

Bellman (expectation) equation: recursion

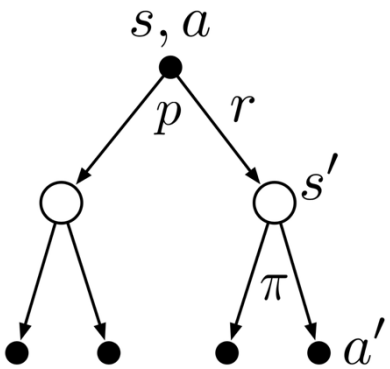
Action-Value Function

$$q_\pi(s_t, a_t) = E_\pi\left[\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} \,|s_t, a_t\right] = E_\pi[r_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1})|s_t, a_t]$$

$$= \sum_{s'_{t+1}, r_{t+1}} p(s'_{t+1}, r_{t+1}|s_t, a_t)\left[r_{t+1} + \gamma \sum_{a'_{t+1}} \pi(a'_{t+1}|s'_{t+1})\, q_\pi(s'_{t+1}, a'_{t+1})\right]$$



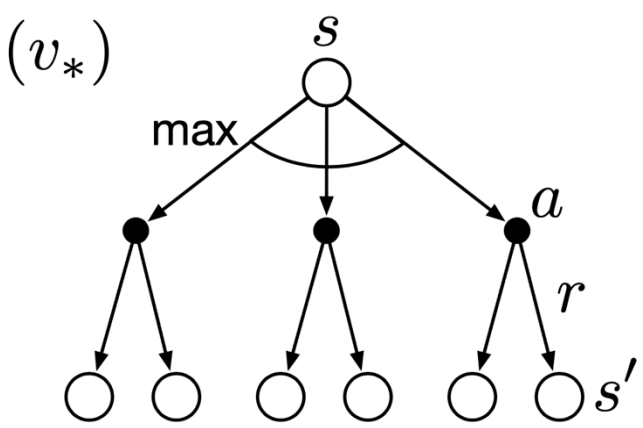
Backup diagram for v_π

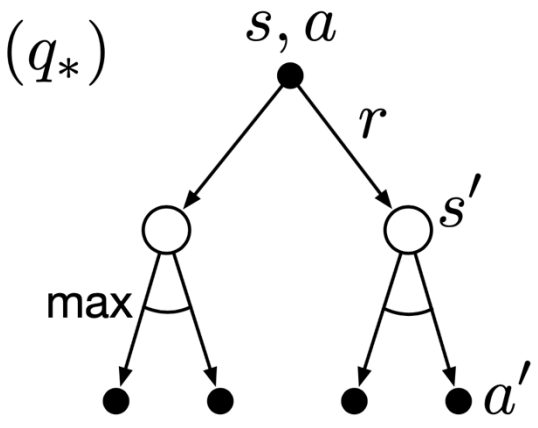


q_π backup diagram

Bellman Optimality Equations

optimal solutions to Bellman equations (directly defining optimal policy):





rarely possible to find in practice (due to missing model of environment, invalid Markov property, limited computational resources)

→ approximate solutions

Planning

Dynamic Programming (DP)

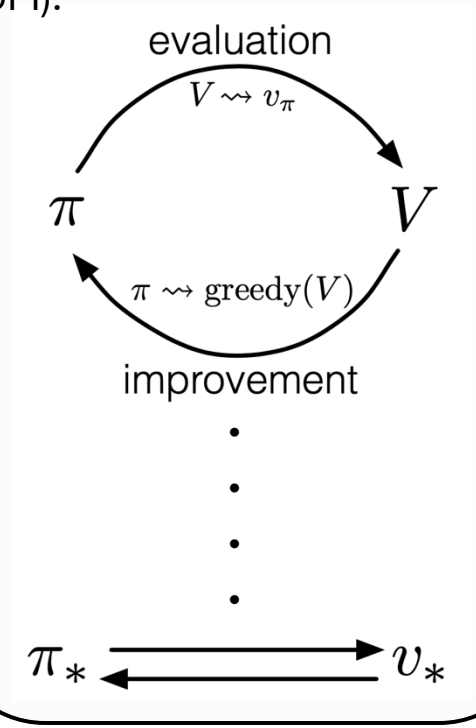
iterative approaches to find approximations for optimal value functions

1. policy evaluation: calculate value function with current policy (Bellman equation as update rule)
2. policy improvement: adjusting policy to act greedy (pick actions with maximum values) with respect to value function of current policy

putting both components together:

- policy iteration: $\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$
- value iteration: truncated policy evaluation using Bellman optimality equation as update rule (stopped after one update of each state)

generalized policy iteration (GPI):



GPI also followed by MC and TD methods …

Limited Utility of DP

requires full model of environment

computationally expensive

- expected update operation (based on values of all possible successor states and their probability)
- for each state (in potentially huge state space)

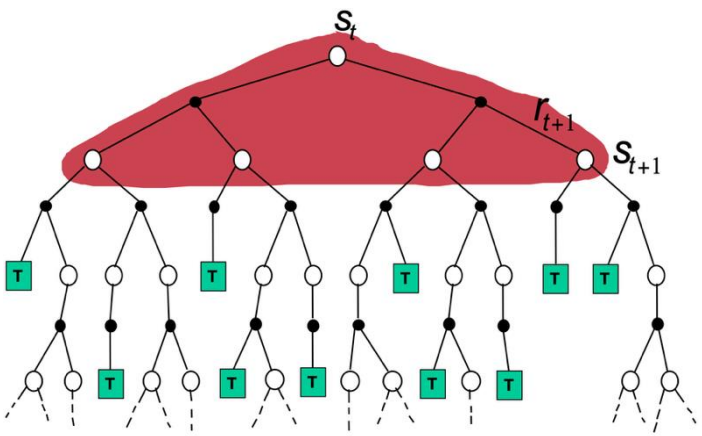
(asynchronous DP at least avoids systematic sweeps over entire state space)

→ need for more efficient methods achieving the same effect as DP, without (perfect) model of environment

Bootstrapping and Sampling

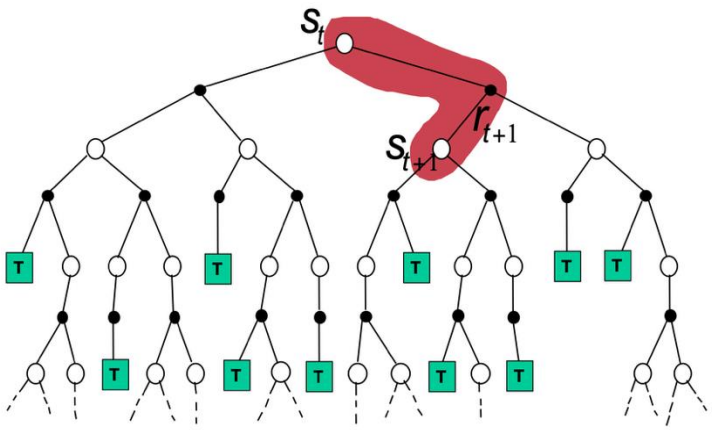
bootstrapping: update estimates of state values based on estimates of values of successor states
sampling: experience of sample sequences (no need for complete knowledge of environment)

Dynamic Programming



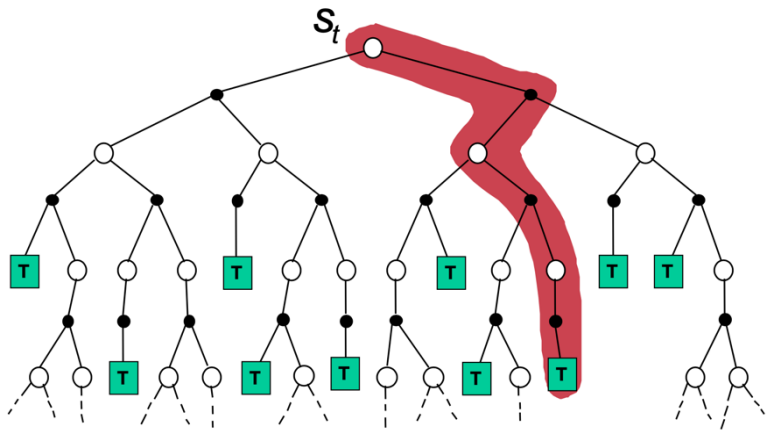
- bootstrapping
- no sampling → model-based (transition probabilities needed)

Temporal Difference (TD) Learning



- bootstrapping
- sampling → model-free

Monte Carlo (MC)



- no bootstrapping
- sampling → model-free

Model-Free Learning

Sampling Update Rule

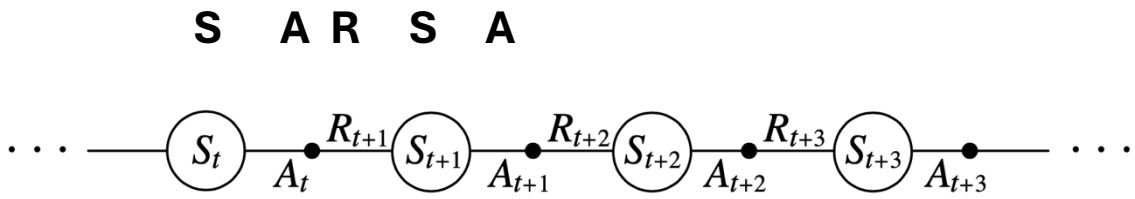
$$\boxed{NewEstimate \leftarrow OldEstimate \; + \; StepSize \left[Target - OldEstimate \right]}$$

MC: $v(s_t) \leftarrow v(s_t) + \eta[\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} - v(s_t)]$

TD: $v(s_t) \leftarrow v(s_t) + \eta[r_{t+1} + \gamma v(s_{t+1}) - v(s_t)]$

bootstrapping

On-Policy TD Control: SARSA



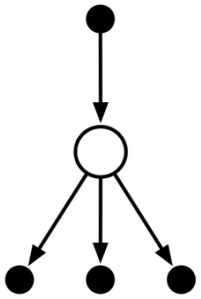
following pattern of GPI:

- estimate action-value function for current behavior policy
$$q_\pi(s_t, a_t) \leftarrow q_\pi(s_t, a_t) + \eta[r_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1}) - q_\pi(s_t, a_t)]$$

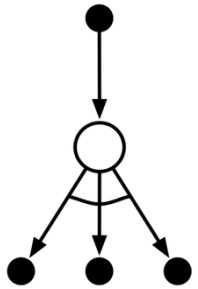
- change policy toward greediness with respect to q_π (exploration for example via ε-greedy policy)

Expected SARSA

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \eta \left[r_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) q(s_{t+1}, a) - q(s_t, a_t) \right]$$



Off-Policy TD Control: Q-Learning



approximation of optimal action-value function:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

behavior policy (used to collect data) can differ from target policy (greedy) → off-policy

Q stands for quality function: estimate of true action-value function

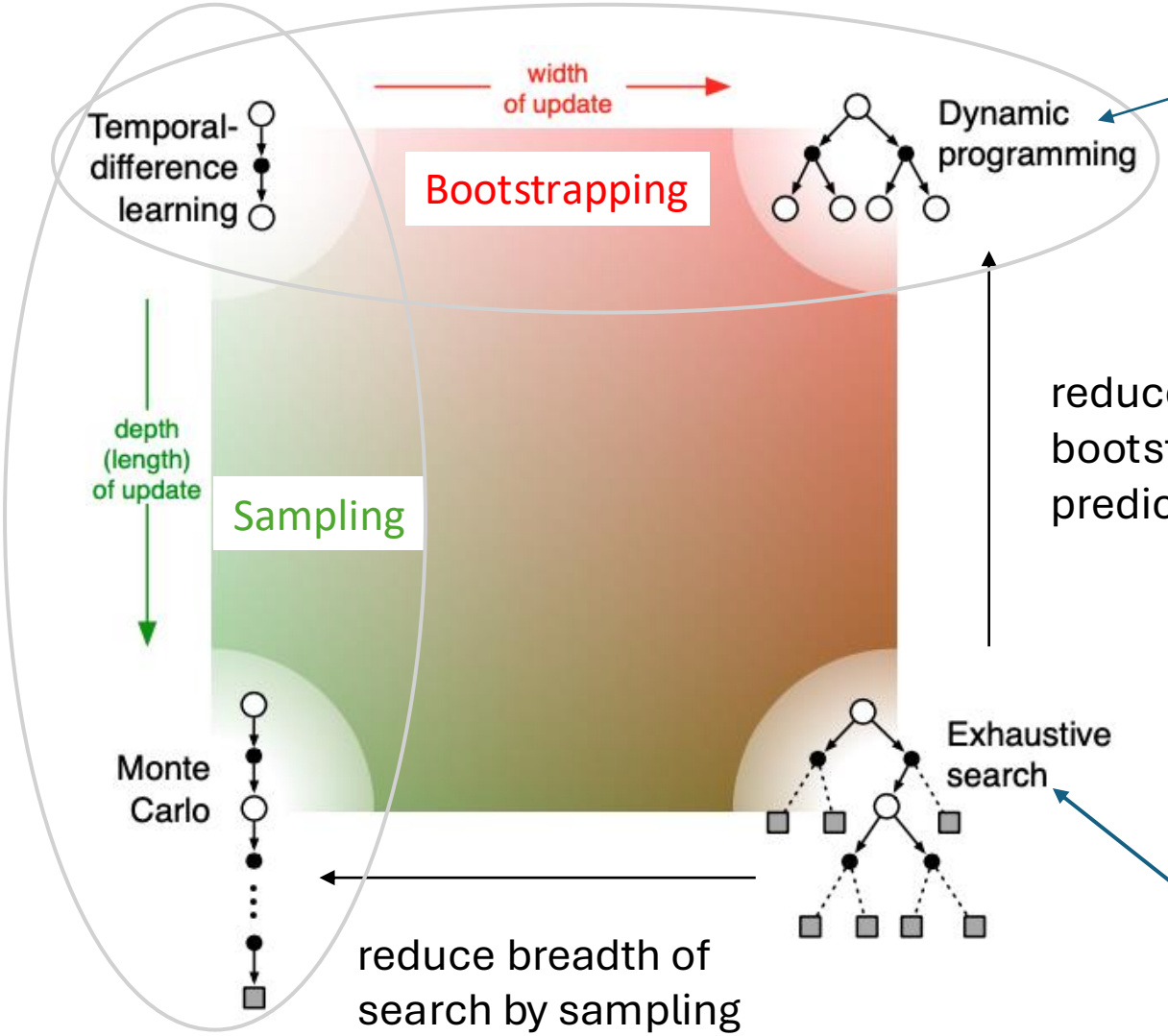
control strategy: choose action with highest Q-value (exploitation)

Update Characteristics

one-step TD

n-step TD

MC



model of environment needed

reduce depth of search by bootstrapping with value function predicting future outcome

reduce breadth of search by sampling actions from policy

very inefficient

Practice Project

Gymnasium Blackjack